

Assignment 3

Submission deadline: 29/06/2026

Defence Dates: 06/07/2026-07/07/2026

Link to defense registration: [📅 Deep Learning Assignemnt Defense Schedule](#)

Part 0: Info

Part 0.1: Questions about the assignment

Questions about the assignment **should be submitted only on the Moodle forum.**

Any questions sent via email that are not related to administrative or personal issues (e.g., extension in submission due to justified reasons) will be ignored without any response.

If you have a question you want to ask, that means five other people might also have the same question but are afraid to ask. Posting questions on Moodle will help not only you but also your classmates.

Part 0.3: Introduction

In this assignment, we will dive deeper into understanding neural networks and specifically, transformer models.

Part 0.5: Using GPUs

As mentioned in the lectures and TA sessions, neural networks benefit from GPUs because they can train faster and more efficiently. To support this, you have access to the CS cluster. Make sure you follow the Moodle guides for accessing and using it.

In your code, make sure your tensors and models are actually running on the GPU (device = cuda) when you intend to use it.

Part 1: Training Autoregressive Language Models Story Generation (20 points)

1. In this section we will train our own little language model to generate short stories. We'll use the [TinyStories dataset](#) for this task.
2. The dataset is provided with the assignment and contains 200,000 stories that can be used to train our model, and 1,000 stories that will be used for validation. You can download the dataset from the following link:
<https://drive.google.com/file/d/1j6BgMSuKvwUqUnyILztdmkA0zcpzLcbC/view?usp=sharing>

Note: The link is only accessible from the university email address, permission requests from other mail addresses will be blocked.

3. You are provided the following files for this task:
 - a. merges.json- This file contains information for the tokenizer which includes: possible merges, special tokens, and splitting pattern.
 - b. tokenizer.py- This file contains a tokenizer called *BPETokenizer*, this file handles converting our texts into tokens. Please take your time reading it to understand it more.
The *BPETokenizer* contains a "load" method to load the merges.json
 - c. model.py- This file contains an implementation of the *GPT* Transformer language model.
 - d. config.toml and config.py- These files contain the model hyper-parameters we can load for our model.
4. In this step, we will design and train a *GPT* model on the *TinyStories* dataset. For each of the following sub steps, describe your implementation, present the results, and discuss your observations.
 - a. Train the *GPT* model on your train-set and use the validation-set for evaluating different hyperparameters
Notes:
 - i. First try to play with other hyper-parameters before you change the model's parameters.
 - ii. Do not change the block size parameter no matter what, changing it will result in point reduction
 - iii. It is recommended to use the `configure_optimizers` method to create your optimizers rather than doing it yourself.
 - b. After you have chosen a set of hyperparameters, generate two line plots: (a) training and validation loss as a function of step, and (b) training and validation accuracy as a function of step.

Note: validation accuracy and validation loss should be computed on the entire validation set.

- c. After the model is trained, use the *generate* method to generate new stories. Generate 5 stories and add them to your report. Are the stories coherent? Are they unique? Do you notice repeating behaviour in the models generated stories?

Part 2: Implementing Relative Positional Encoding For Extending Context length(50 points)

1. In this section we'll expand our transformer model using *Relative Positional Encoding*. *Relative Positional Encoding* allows *Transformer* models to expand to longer sequences than those they were trained on.
2. Please read the following papers:
 - a. [Self-Attention with Relative Position Representations](#)
 - b. [Music Transformer: Generating Music with Long-Term Structure](#)

These papers contain information on *Relative Positional Encoding* and how they were implemented first in Transformers.

Our goal is to implement the relative encoding in paper (a), paper (b) gives us an option to write a more efficient implementation.

3. For this task you are supplied with a new file `model_relative.py`. This file contains the same code as in our `model.py` but with small modifications to help us later (such as removing the `wpe`, and any assertions about block size).
4. Implement the *Relative Positional Encoding* in our model and repeat the entirety of step 4 from part 1.
 - a. In the config, there is already a parameter `k` set for you to use for your implementation.
K sets the max distance of the relative distance we learn between tokens.
 - b. Update the `CausalAttention` module with your implementation of the relative positional encoding. Your changes should only be in this module; if you need to change somewhere else in the code, you are doing something wrong.
NOTE: Please implement relative encoding only for the *key vectors*. Do not add relative encoding for the *value vectors*

$$\text{RelativeAttention} = \text{Softmax} \left(\frac{QK^\top + S_{rel}}{\sqrt{D_h}} \right) V \quad (3)$$

5. In this step, we want to test whether the models can use information that appeared earlier in the text when predicting a later token. This is especially important when the relevant information appears farther away than the GPT context window.
 - a. Load the examples from `long_context_recall_stories.jsonl`. Each example contains a TinyStories-style prompt, an expected answer token, and the distance

between the original fact and the answer position.

- b. Evaluate both models on each example:
 - i. Absolute positional encoding GPT
 - ii. Relative positional encoding GPT
- c. For each model and each example, compute the loss of the correct answer token at the answer position.
- d. Create a single line plot comparing the two models:
 - x-axis: distance between the original fact and the answer position
 - y-axis: loss of the correct answer token
- e. Add the plot to your report and discuss your observations. In your discussion, answer:
 - Does the absolute-position GPT lose performance when the required information is far away?
 - Does the relative-position GPT behave differently?
 - How does your chosen value of K affect the result?
 - Does relative positional encoding fully solve long-context recall? Explain.

Notes:

- Do not change `block_size`.
- Use the trained models in evaluation mode.
- This evaluation measures the model's loss on the correct next token, not the quality of a freely generated story.

6. BONUS: best model submitted in part 2 from all the groups, evaluated by the lowest loss achieved on the validation set will receive a bonus of 3 points to the assignment.

So please report numerically the final validation loss of the model on the entire set. I will not infer loss values for the figures and graphs, so a group without explicitly writing the validation loss will not get to participate in the bonus.

Note: only five groups will get the bonus.

Part 3: Evaluating Autoregressive Language Models for Grammatical Acceptability (15 points)

1. This section investigates the ability of autoregressive language models to predict human judgments of grammatical acceptability. We will utilize the CoLA (Corpus of Linguistic Acceptability) dataset, available through the GLUE benchmark (<https://huggingface.co/datasets/nyu-mll/glue/viewer/cola/>).
2. Our analysis will begin with the "openai-community/gpt2" model, the smallest variant of GPT-2, sourced from Hugging Face (<https://huggingface.co/openai-community/gpt2>). The primary objective is to determine if the sentence probabilities assigned by the model correlate with human assessments of acceptability.
3. The following steps will use the CoLA validation set as it will serve as our test set, as the official test set lacks labels, if you want to create a validation set, consider splitting the train set to train-validation.
4. In this section, we will evaluate whether the probability assigned to a sentence by the model predicts how *acceptable* the sentence is to human English speakers.

Convert these logits into log-softmax probabilities. For each token position, identify the log-probability corresponding to the actual token in the input sentence.

Calculate the overall log-probability of the sentence by summing the log-probabilities of the correct tokens across all positions.

Generative Transformers are compatible with language modeling, in which a model assigns a likelihood to a sequence of words ($S = [W_1, \dots, W_n]$). Transformers estimate the likelihood of a sequence by factoring on sequence order:

$$P(S) = \prod_{i=1}^N P(W_i | W_1 \dots W_{i-1}) \quad (1)$$

Taking the log of the likelihood ($\log(P(S))$) is often preferred as it allows for numerical stability.

5. Performance Evaluation (ROC AUC): Using the calculated sentence log-probabilities as the `y_score` and the human acceptability judgments (true labels) as `y_true`, compute the `roc_auc_score`.

Note: that ROC AUC scores above 0.5 indicate better-than-random performance. If your scores are consistently below 0.45, consider multiplying the `y_score` by -1.

6. Repeat steps 1-4 for the gpt2-medium, gpt2-large, and gpt2-xl models.

Generate a line graph illustrating the ROC AUC score as a function of GPT model size. The x-axis must explicitly display the model names (e.g., "gpt2", "gpt2-medium", "gpt2-large", "gpt2-xl"). This graph will demonstrate the relationship between model size and its predictive accuracy of human acceptability judgments.

IMPORTANT: The graph must contain the model name on the x-axis and not anything else.

7. Repeat steps 1-5, However, instead of summing the log-softmax probabilities (as in step 3), calculate the average log-softmax probability by dividing the sum by the number of tokens in the sentence (i.e., normalize by sequence length).
8. Determine which method, summation or averaging of log-probabilities, yielded superior performance. Provide a reasoned explanation for why one approach might be more effective than the other in this context.

Part 4: Fine-Tuning a Model on CoLA (15 points)

Here, you'll fine-tune the previously used GPT models to classify sentence acceptability on the CoLA dataset.

1. Choose & Load Model: Pick a GPT model from Part 3. Load it using `AutoModelForSequenceClassification` from Hugging Face.
2. Prepare Data & Fine-Tune:
 - a. Take the CoLA train data.

Important: Split this train data into its own train and validation subsets. Use this new validation subset for hyperparameter tuning while you fine-tune the model on the new train subset.

3. Test: After fine-tuning, use the fine-tuned model to make predictions on the main CoLA validation set. (Remember, this is our test set since the actual test set has no labels).

Submission Instructions:

Please read all the instructions carefully and make sure you meet all the requirements before submitting.

1. Submissions are done in **PAIRS** unless you have special permission.
2. Only **one person** from the pair submits the assignment.
3. The assignment should be a single zip file named ID1_ID2.zip (ID1 and ID2 are the actual students)
4. The assignment should be a single zip file containing the following:
 - a. A PDF file containing graphs and answers to questions
 - b. A single .ipynb notebook file OR .py files including all the experiments you ran.
 - c. Output of the models' generated stories in txt files. One per model.
5. The notebook/python file should be well organized, follow good coding standards, and be easy to read.